

Phase 1 - Week 1 - Lesson Day 1

Python Refresh for Data Science

Today's Objectives

You already know Python from full-stack development. Today isn't about learning Python from zero - it's about seeing the tools you already know (lists, dicts, functions) through a **data science lens**, and getting comfortable working inside a notebook environment, which is the standard way data scientists write and explore code.

- Recap core Python data types and where they show up in data work
- Understand and write list comprehensions
- Understand and write lambda functions
- Understand why data scientists use Jupyter/Colab notebooks instead of plain .py files

1. Data Types Recap (Data Science Context)

Definition: A *data structure* is simply a way of organizing data so it can be stored and used efficiently. In data science, you constantly convert between these basic Python structures and specialized ones like NumPy arrays or Pandas DataFrames - so being fluent in the basics matters.

Type	Example	Where it matters in Data Science
List	[1, 2, 3]	Storing raw data before converting to arrays
Dictionary	{"name": "Ali", "age": 25}	Representing a single record (like a JSON row from an API)
Tuple	(1, 2)	Fixed, unchangeable data (e.g. coordinates, fixed configs)
Set	{1, 2, 3}	Quickly removing duplicates from data

2. List Comprehensions

Definition: A *list comprehension* is a compact way to build a new list by applying an expression to every item in an existing iterable (like a list or range), optionally filtering items - all written in a single line instead of a multi-line loop.

The traditional way, using a loop:

```
squares = []
for x in range(10):
    squares.append(x**2)
```

The list comprehension way (identical result, one line):

```
squares = [x**2 for x in range(10)]
```

Why it matters in data science: you'll use this constantly to quickly transform raw data before it goes into a model - for example, converting a list of raw prices into a list of prices with tax applied, in one clean line.

You can also filter while building the list:

```
evens = [x for x in range(20) if x % 2 == 0]
```

Read this as: *"give me x, for every x in range(20), but only if x is divisible by 2."* The **if** at the end is the filter condition.

3. Lambda Functions

Definition: A *lambda function* is a small, unnamed (anonymous) function written in a single line, typically used for short, throwaway operations where defining a full function with **def** would feel like overkill.

```
square = lambda x: x**2  
print(square(5))    # 25
```

This is functionally identical to:

```
def square(x):  
    return x**2
```

Why it matters in data science: lambdas are used heavily with **map()**, **filter()**, and later with Pandas' **.apply()** to transform columns of data on the fly, without writing a separate named function for every small transformation.

map() - applies a function to every item in a list:

```
numbers = [1, 2, 3, 4]  
doubled = list(map(lambda x: x * 2, numbers))  
# [2, 4, 6, 8]
```

filter() - keeps only items where the function returns True:

```
numbers = [1, 2, 3, 4, 5, 6]  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
# [2, 4, 6]
```

4. Jupyter Notebook / Google Colab

Definition: These are *interactive coding environments* where you write and run code in small chunks called *cells*, seeing the output immediately below each cell - instead of running an entire .py file at once from top to bottom.

Why data scientists use this instead of plain .py files: data work is exploratory. You're constantly checking what a dataset looks like, testing one transformation, plotting a quick chart, then deciding the next step based on what you saw. Re-running an entire script every time (as is normal in software engineering) would be painfully slow and wasteful.

Since you're both coming from full-stack development, think of a notebook like a live REPL that remembers your variables across cells, and lets you mix code, output, charts, and written notes in a single scrollable document. Google Colab (colab.research.google.com) is a free, browser-based version - no installation needed, which is what we'll use for this course.

Today's Exercise

Open the accompanying notebook file **day-1-python-refresh.ipynb** and complete the exercises below inside it. Each one maps directly to a concept covered above - read the explanation of what's being asked, then write the code yourself in the notebook before checking back here.

Part A - List Comprehensions (5 total)

1. Squares from 1-10

Build a list containing the square of every number from 1 to 10. This is the exact pattern shown in the lesson - one expression, one loop variable, no filter needed.

2. Filter even numbers (1-50)

Loop through numbers 1 to 50 and keep only the ones divisible by 2. This uses the comprehension-with-a-filter pattern from the lesson (the trailing **if**).

3. Word lengths

Given **["data", "science", "AI", "python"]**, build a new list containing the length of each word. Think about which built-in function returns the length of a string.

4. Celsius to Fahrenheit

Given **[0, 10, 20, 30]**, build a new list converting each value using the formula $(C * 9/5) + 32$. This is a pure transformation, no filtering needed.

5. Words starting with a vowel

Given **["apple", "banana", "orange", "grape", "egg"]**, keep only the words whose first letter is a vowel (a, e, i, o, u). You will need to check the first character of each word inside the **if** condition.

Part B - Lambda Functions (3 total)

1. Cube with map()

Write a lambda function that cubes a number (x^{**3}), then use `map()` to apply it to `[1, 2, 3, 4]` and convert the result to a list.

2. Remove negatives with filter()

Write a lambda that returns `True` only for positive numbers, then use `filter()` to remove all negative numbers from `[-5, 3, -2, 8, -1, 10]`.

3. Long words with filter()

Write a lambda that checks whether a string has more than 5 characters, then use `filter()` to keep only the long words from `["cat", "elephant", "dog", "python", "ai"]`.

How to Approach It

For every exercise: first identify what the **loop variable** is, what the **expression** (the transformation) is, and whether a **filter condition** is needed. If you get stuck, write the slow version first (a normal for-loop with `.append()`), get it working, then compress it into a one-line comprehension or lambda - that's exactly how experienced data scientists debug these when they don't come out right the first time.

Once you and Halimo have both completed the exercises in the notebook, bring your answers back to the lesson chat and we'll review them together before moving on to Lesson Day 2 (NumPy Basics).